

Bazy i hurtownie danych

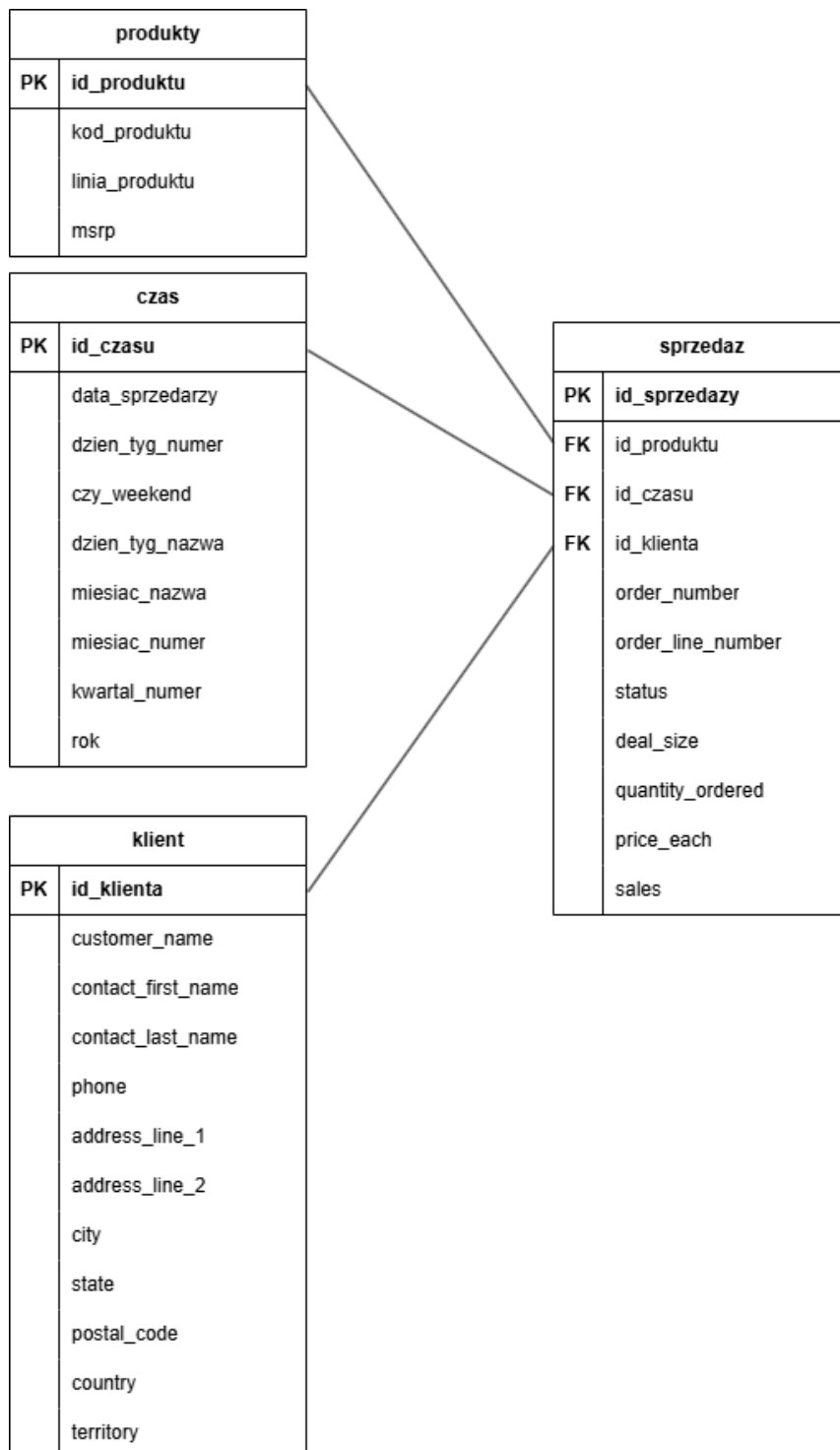
Projekt końcowy

Piotr Kardyński, Jan Chrzanowski

Oświadczamy, że poniższe zadania wykonaliśmy samodzielnie bez pomocy osób trzecich, czy też narzędzi AI. Oprócz: W zadaniu 2 AI zostało użyte do wytłumaczenia niektórych elementów programu Power Bi, interpretacji zadania z perspektywą zmaterializowaną oraz w zadaniu 1 w poprawieniu klauzuli CASE WHEN TO_CHAR(s.czysta_data, 'D') IN ('6', '7') THEN 'TAK' ELSE 'NIE' END w MERGE INTO czas - wcześniej występował błąd z powodu złego zapisu i pomocy w usuwaniu danych (słowo CASCADE).

Projekt był realizowany z użyciem github(link do wszystkich plików):
https://github.com/Pioxe/Bazy_danych_projekt

Zadanie 1:

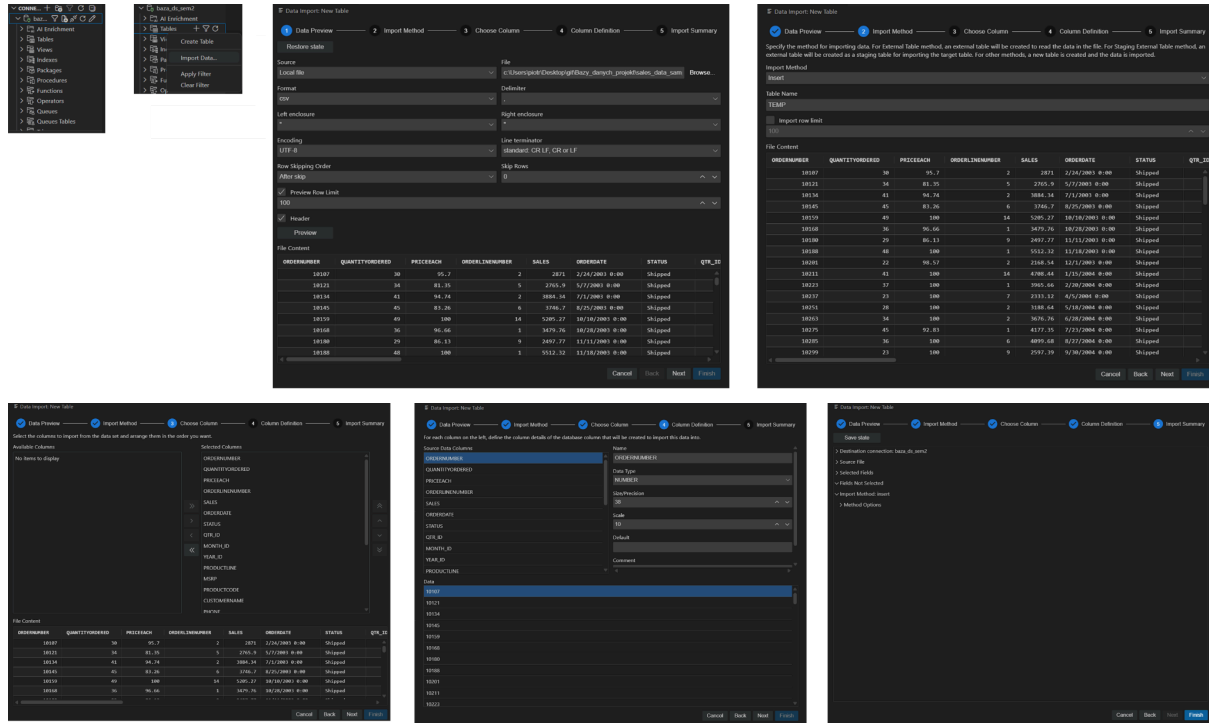


Import zbioru do TEMP:

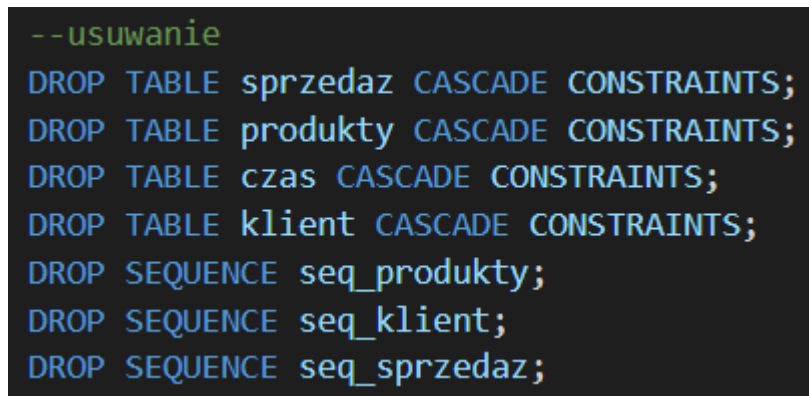
Zbiór został zaimportowany dzięki wtyczce Oracle SQL Developer, podczas importu należy się upewnić czy wtyczka poprawnie rozpoznała dane - np ORDERDATE, w csv jest zapisana w formacie tekstowym - 2/24/2003 00:00 - więc trzeba uważać by wtyczka omyłkowo nie wypełniła nam pola ORDERDATE typem DATE. Kolejnym problemem jest

nieznajomość długości wartości, by go rozwiązać trzeba dać duże wartości w nawiasach koło typu danych, a gdy już wpisujemy tabele TEMP i będziemy znać długości danych, mergując dane do tabel wielkość np VARCHAR2 zmniejszymy.

Poniżej poglądowe zrzuty ekranu importu danych:



Po zaimportowaniu danych przygotowujemy tabele: produkty, czas, sprzedaz, klient. Jednak przed tym usuwamy pozostałości po tabelach które możemy mieć w bazie danych.



Polecenia tworzenia tabel:

DO wszystkich tabel wybieramy ograniczenia danych np 'CHECK (czy_weekend IN ('TAK', 'NIE')) NOT NULL' i typy danych. Po utworzeniu tabel wykonujemy kod pobierający i wstawiający dane z tabeli TEMP.

```
-- TABELA WYMIARU: klient
CREATE TABLE klient (
  id_klienta      NUMBER(10) PRIMARY KEY,
  customer_name   VARCHAR2(100) NOT NULL,
  contact_first_name VARCHAR2(50),
  contact_last_name VARCHAR2(50),
  phone           VARCHAR2(30),
  address_line_1  VARCHAR2(150),
  address_line_2  VARCHAR2(150),
  city            VARCHAR2(50),
  state           VARCHAR2(50),
  postal_code     VARCHAR2(30),
  country         VARCHAR2(50),
  territory       VARCHAR2(20)
);

-- TABELA FAKTÓW: sprzedaz
CREATE TABLE sprzedaz (
  id_sprzedazy    NUMBER(10) PRIMARY KEY,
  id_produktu     NUMBER(10) REFERENCES produkty(id_produktu) NOT NULL,
  id_czasu        NUMBER(10) REFERENCES czas(id_czasu) NOT NULL,
  id_klienta      NUMBER(10) REFERENCES klient(id_klienta) NOT NULL,
  order_number    NUMBER(10) NOT NULL,
  order_line_number NUMBER(3) NOT NULL,
  status          VARCHAR2(20),
  deal_size       VARCHAR2(20),
  quantity_ordered NUMBER(6) NOT NULL,
  price_each      NUMBER(10,2) NOT NULL,
  sales           NUMBER(12,2) NOT NULL
);
commit;
```

```
-- TABELA WYMIARU: produkty
CREATE TABLE produkty (
  id_produktu     NUMBER(10) PRIMARY KEY,
  kod_produktu    VARCHAR2(30) NOT NULL,
  linia_produktu  VARCHAR2(50) NOT NULL,
  msrp            NUMBER(10,2)
);

-- TABELA WYMIARU: czas
CREATE TABLE czas (
  id_czasu        NUMBER(10) PRIMARY KEY,
  data_sprzedazy  DATE NOT NULL,
  dzien_tyg_numer NUMBER(2) NOT NULL,
  czy_weekend     VARCHAR2(3) CHECK (czy_weekend IN ('TAK', 'NIE')) NOT NULL,
  dzien_tyg_nazwa VARCHAR2(20) NOT NULL,
  miesiac_nazwa   VARCHAR2(20) NOT NULL,
  miesiac_numer   NUMBER(2) NOT NULL,
  kwartal_numer   NUMBER(1) NOT NULL,
  rok             NUMBER(4) NOT NULL
);
```

```
14
15  -- TABELA WYMIARU: produkty
16  CREATE TABLE produkty (
17    id_produktu     NUMBER(10) PRIMARY KEY,
18    kod_produktu    VARCHAR2(30) NOT NULL,
19    linia_produktu  VARCHAR2(50) NOT NULL,
20    msrp            NUMBER(10,2)
21  );
22
23  -- TABELA WYMIARU: czas
24  CREATE TABLE czas (
25    id_czasu        NUMBER(10) PRIMARY KEY,
26    data_sprzedazy  DATE NOT NULL,
27    dzien_tyg_numer NUMBER(2) NOT NULL,
28    czy_weekend     VARCHAR2(3) CHECK (czy_weekend IN ('TAK', 'NIE')) NOT NULL,
29    dzien_tyg_nazwa VARCHAR2(20) NOT NULL,
30    miesiac_nazwa   VARCHAR2(20) NOT NULL,
31    miesiac_numer   NUMBER(2) NOT NULL,
32    kwartal_numer   NUMBER(1) NOT NULL,
33    rok             NUMBER(4) NOT NULL
34  );
35
36  -- TABELA FAKTÓW: sprzedaz
37  CREATE TABLE sprzedaz (
38    id_sprzedazy    NUMBER(10) PRIMARY KEY,
39    id_produktu     NUMBER(10) REFERENCES produkty(id_produktu) NOT NULL,
40    id_czasu        NUMBER(10) REFERENCES czas(id_czasu) NOT NULL,
41    id_klienta      NUMBER(10) REFERENCES klient(id_klienta) NOT NULL,
42    order_number    NUMBER(10) NOT NULL,
43    order_line_number NUMBER(3) NOT NULL,
44    status          VARCHAR2(20),
45    deal_size       VARCHAR2(20),
46    quantity_ordered NUMBER(6) NOT NULL,
47    price_each      NUMBER(10,2) NOT NULL,
48    sales           NUMBER(12,2) NOT NULL
49  );
50
51  commit;
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULT SCRIPT OUTPUT SQL HISTOR

Table PRODUKTY created.

Table CZAS created.

Table KLIENT created.

Table SPRZEDAZ created.

Commit complete.

```

MERGE INTO czas t
USING (
    SELECT DISTINCT TRUNC(TO_DATE(orderdate, 'MM/DD/YYYY HH24:MI')) AS czysta_data
    FROM temp
) s
ON (t.id_czasu = TO_NUMBER(TO_CHAR(s.czysta_data, 'YYYYMMDD')))
WHEN NOT MATCHED THEN
    INSERT (id_czasu, data_sprzedazy, dzien_tyg_numer, czy_weekend, dzien_tyg_nazwa, miesiac_nazwa, miesiac_numer, kwartal_numer, rok)
    VALUES (
        TO_NUMBER(TO_CHAR(s.czysta_data, 'YYYYMMDD')),
        s.czysta_data,
        TO_NUMBER(TO_CHAR(s.czysta_data, 'D')),
        CASE WHEN TO_CHAR(s.czysta_data, 'D') IN ('6', '7') THEN 'TAK' ELSE 'NIE' END,
        TO_CHAR(s.czysta_data, 'Day'),
        TO_CHAR(s.czysta_data, 'Month'),
        TO_NUMBER(TO_CHAR(s.czysta_data, 'MM')),
        TO_NUMBER(TO_CHAR(s.czysta_data, 'Q')),
        TO_NUMBER(TO_CHAR(s.czysta_data, 'YYYY'))
    );

COMMIT;

CREATE SEQUENCE seq_klient
START WITH 1
INCREMENT BY 1
NOCACHE
NOCYCLE;

MERGE INTO klient k
USING (
    SELECT DISTINCT
        customername, contactfirstname, contactlastname, phone, addressline1, addressline2, city, state,
        postalcode, country, territory
    FROM temp
    WHERE customername IS NOT NULL
) s
ON (k.customer_name = s.customername)
-- czy klient już istnieje
WHEN NOT MATCHED THEN
    INSERT (
        id_klienta, customer_name, contact_first_name, contact_last_name, phone, address_line_1, address_line_2, city, state,
        postal_code, country, territory
    )
    VALUES (seq_klient.NEXTVAL, s.customername, s.contactfirstname, s.contactlastname, s.phone, s.addressline1, s.addressline2,
        s.city, s.state, s.postalcode, s.country, s.territory);

```

```

CREATE SEQUENCE seq_produkty
START WITH 1
INCREMENT BY 1
NOCACHE;

MERGE INTO produkty t
USING (
    SELECT DISTINCT productcode, productline, msrp
    FROM temp
) s
ON (t.kod_produkту = s.productcode)

WHEN MATCHED THEN
    UPDATE SET
        t.linia_produkту = s.productline,
        t.msrp = s.msrp

WHEN NOT MATCHED THEN
    INSERT (id_produkту, kod_produkту, linia_produkту, msrp)
    VALUES (seq_produkty.NEXTVAL, s.productcode, s.productline, s.msrp);

CREATE SEQUENCE seq_sprzedaz
START WITH 1
INCREMENT BY 1;

MERGE INTO sprzedaz s
USING (
    SELECT
        p.id_produkту, c.id_czasu, k.id_klienta, t.ordernumber, t.orderlinenumber, t.status, t.dealsize, t.quantityordered,
        t.priceeach, t.sales
    FROM temp t
    JOIN produkty p
        ON p.kod_produkту = t.productcode
    JOIN klient k
        ON k.customer_name = t.customername
    JOIN czas c
        ON c.data_sprzedazy = TO_DATE(t.orderdate, 'MM/DD/YYYY HH24:MI')
) src
ON (
    s.order_number = src.ordernumber
    AND s.order_line_number = src.orderlinenumber
)
WHEN NOT MATCHED THEN
    INSERT (
        id_sprzedazy, id_produkту, id_czasu, id_klienta, order_number, order_line_number, status, deal_size, quantity_ordered, price_each, sales
    )
    VALUES (
        seq_sprzedaz.NEXTVAL, src.id_produkту, src.id_czasu, src.id_klienta, src.ordernumber, src.orderlinenumber, src.status, src.dealsize,
        src.quantityordered, src.priceeach, src.sales );

```

```
72      INCREMENT BY 1
73      NOCACHE;
74
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Sequence SEQ_PRODUKTY created.

109 rows merged.

252 rows merged.

Commit complete.

Sequence SEQ_KLIENT created.

92 rows merged.

Commit complete.

Sequence SEQ_SPRZEDAZ created.

2,823 rows merged.

Commit complete.
```

Zdjęcia z mergami zostały “zlepione” w programie graficznym (tak samo create table) by więcej się na nich mieściło temu wywołania nie są po kolei w konsoli

MERGE INTO czas: Jest to polecenie które obsługuje rozdzielenie czasu z tabeli temp na różne kolumny.

MERGE INTO produkty: ten blok kodu wstawia kod_produkту, linia_produkту oraz msrp.

Nad nim znajduje się również blok z SEQUENCE, blok ten generuje id dla tej tabeli.

Dodatkowo jeśli kod produktu się powtarza to w merge jest klauzula która wybiera i updatuje linia_produkту oraz msrp na nowszą wartość.

MERGE INTO sprzedaz: wstawia informacje do tabeli sprzedaz, również korzysta z generowania id z seq_sprzedaz podobnie jak tabela produkty.

MERGE INTO klient: merge do klienta również nie jest skomplikowany - jest bardzo podobny do merge w tabeli sprzedaz.

Wyniki count(*):

	TABELA	COUNT(*)
1	produkty	109
2	czas	252
3	klient	92
4	sprzedaz	2823

gdy porównamy tabele sprzedaz z tabelą temp zobaczymy że ilość rekordów się zgadza:

	'TEMP'	COUNT(*)
1	temp	2823

10 Pierwszych wierszy z każdej tabeli:

Produkty:

	ID_PRODUKTU	KOD_PRODUKTU	LINIA_PRODUKTU	MSRP
1	1	S10_1678	Motorcycles	95
2	2	S10_1949	Classic Cars	214
3	3	S10_2016	Motorcycles	118
4	4	S10_4698	Motorcycles	193
5	5	S10_4757	Classic Cars	136
6	6	S10_4962	Classic Cars	147
7	7	S12_1099	Classic Cars	194
8	8	S12_1108	Classic Cars	207
9	9	S12_1666	Trucks and Buses	136
10	10	S12_2823	Motorcycles	150

Sprzedaż:

	ID_SPRZEDAZY	ID_PRODUKTU	ID_CZASU	ID_KLIENTA	ORDER_NUMBER	ORDER_LINE_NUMBER	STATUS	DEAL_SIZE	QUANTITY_ORDERED	PRICE_EACH	SALES
1	527	11	20050505	11	10413	3	Shipped	Large	47	100	8236.75
2	528	68	20041112	8	10328	1	Shipped	Small	48	58.92	2828.16
3	529	95	20030507	35	10121	1	Shipped	Medium	44	74.85	3293.4
4	530	11	20050217	85	10382	11	Shipped	Medium	37	100	4071.85
5	531	84	20040819	49	10281	8	Shipped	Small	29	82.83	2402.07
6	532	86	20040526	18	10252	6	Shipped	Small	36	48.28	1739.08
7	533	4	20030825	28	10145	8	Shipped	Medium	33	100	5176.38
8	534	40	20030801	39	10141	9	Shipped	Medium	34	100	4836.5
9	535	103	20050131	60	10373	16	Shipped	Small	41	70.33	2883.53
10	536	27	20031121	83	10193	13	Shipped	Small	42	59.33	2491.86

Czas:

	ID_CZASU	DATA_SPRZEDAZY	DZIEŃ_TYG_NUMER	CZY_WEEKEND	DZIEŃ_TYG_NAZWA	MIESIAC_NAZWA	MIESIAC_NUMER	KWARTAL_NUMER	ROK
1	20050403	05/04/03	7	TAK	Sunday	April	4	2	2005
2	20030701	03/07/01	2	NIE	Tuesday	July	7	3	2003
3	20040226	04/02/26	4	NIE	Thursday	February	2	1	2004
4	20050209	05/02/09	3	NIE	Wednesday	February	2	1	2005
5	20050315	05/03/15	2	NIE	Tuesday	March	3	1	2005
6	20030411	03/04/11	5	NIE	Friday	April	4	2	2003
7	20031002	03/10/02	4	NIE	Thursday	October	10	4	2003
8	20041014	04/10/14	4	NIE	Thursday	October	10	4	2004
9	20041119	04/11/19	5	NIE	Friday	November	11	4	2004
10	20050423	05/04/23	6	TAK	Saturday	April	4	2	2005

Klient:

	ID_KLIENTA	CUSTOMER_NAME	CONTACT_FIRST_NAME	CONTACT_LAST_NAME	PHONE	ADDRESS_LINE_1	ADDRESS_LINE_2	CITY	STATE	POSTAL_CODE	COUNTRY	TERRITORY
1	1	Petit Auto	Catherine	Dewey	(02) 5554 67	Rue Joseph-Bens 532	(null)	Bruxelles	(null)	B-1180	Belgium	EMEA
2	2	CAF Imports	Jesus	Fernandez	+34 913 728 555	Merchants House, 27-30 Merchant's Quay	(null)	Madrid	(null)	28023	Spain	EMEA
3	3	Mini Caravay	Frederique	Citeaux	88.68.1555	24, place Kluber	(null)	Strasbourg	(null)	67000	France	EMEA
4	4	Alpha Cognac	Annette	Roulet	61.77.6555	1 rue Alsace-Lorraine	(null)	Toulouse	(null)	31000	France	EMEA
5	5	Herku Gifts	veysel	Octan	+47 2267 3215	Drømmen 121, PK 744 Sentrum	(null)	Bergen	(null)	N 5004	Norway	EMEA
6	6	Royale Belge	Pascale	Cetrain	(091) 23 67 2555	Boulevard Viroo, 255	(null)	Charleroi	(null)	B-1300	Belgium	EMEA
7	7	Mini Classics	Steve	Frick	916556452	3750 North Pondale street	(null)	White Plains	NY	10607	USA	NA
8	8	Novelli Gifts	Giovanni	Novelli	035-640555	Via Ludovico il Moro 22	(null)	Bergamo	(null)	24100	Italy	EMEA
9	9	Nv Stores, Co.	Victoria	Ashworth	(171) 555-1555	Fauntleroy Circus	(null)	Manchester	(null)	M1 2WT	UK	EMEA
10	10	Cruz & Sons Co.	Arnold	Cruz	+63 2 555 3587	15 McCallum Street - Nabstet Center #13-03	(null)	Nakati City	(null)	1227 MH	Philippines	Japan

Zadanie 2:

W zadaniu proszono o zrzut polecenia tworzącego perspektywę zmaterializowaną, jednak bez żadnych konkretnów co ma przedstawiać. Stworzony zmaterializowany widok powinien przedstawiać rok, kraj, linie produktu oraz sumę sprzedaży jednak ze względu na zabezpieczenia bazy danych, komenda się nie wykonuje.

```
311
312 CREATE MATERIALIZED VIEW podsumowanie_sprzedazy AS
313 SELECT
314     c.rok,
315     k.country,
316     p.linia_produkту,
317     SUM(s.sales) AS suma_sprzedazy
318 FROM sprzedaz s
319 JOIN czas c ON s.id_czasu = c.id_czasu
320 JOIN klient k ON s.id_klienta = k.id_klienta
321 JOIN produkty p ON s.id_produkту = p.id_produkту
322 GROUP BY c.rok, k.country, p.linia_produkту;
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SCRIPT OUTPUT SQL HISTORY

```

SUM(s.sales) AS suma_sprzedazy
FROM sprzedaz s
JOIN czas c ON s.id_czasu = c.id_czasu
JOIN klient k ON s.id_klienta = k.id_klienta
JOIN produkty p ON s.id_produkту = p.id_produkту
GROUP BY c.rok, k.country, p.linia_produkту
Error report -
ORA-01031: insufficient privileges

https://docs.oracle.com/error-help/db/ora-01031/
01031. 00000 - "insufficient privileges"
*Cause:      A database operation was attempted without the required
              privilege(s).
*Action:     Ask your database administrator or security administrator to grant
              you the required privilege(s).
```

Podobnie jak przy poprzednim wymaganiu, tu też rzut analitycznego zapytania wraz z danymi, które zwraca nie miał żadnych konkretów. W poniższym zapytaniu zwracamy sumę sprzedaży wraz z rankingiem podzielonym na poszczególne lata.

```

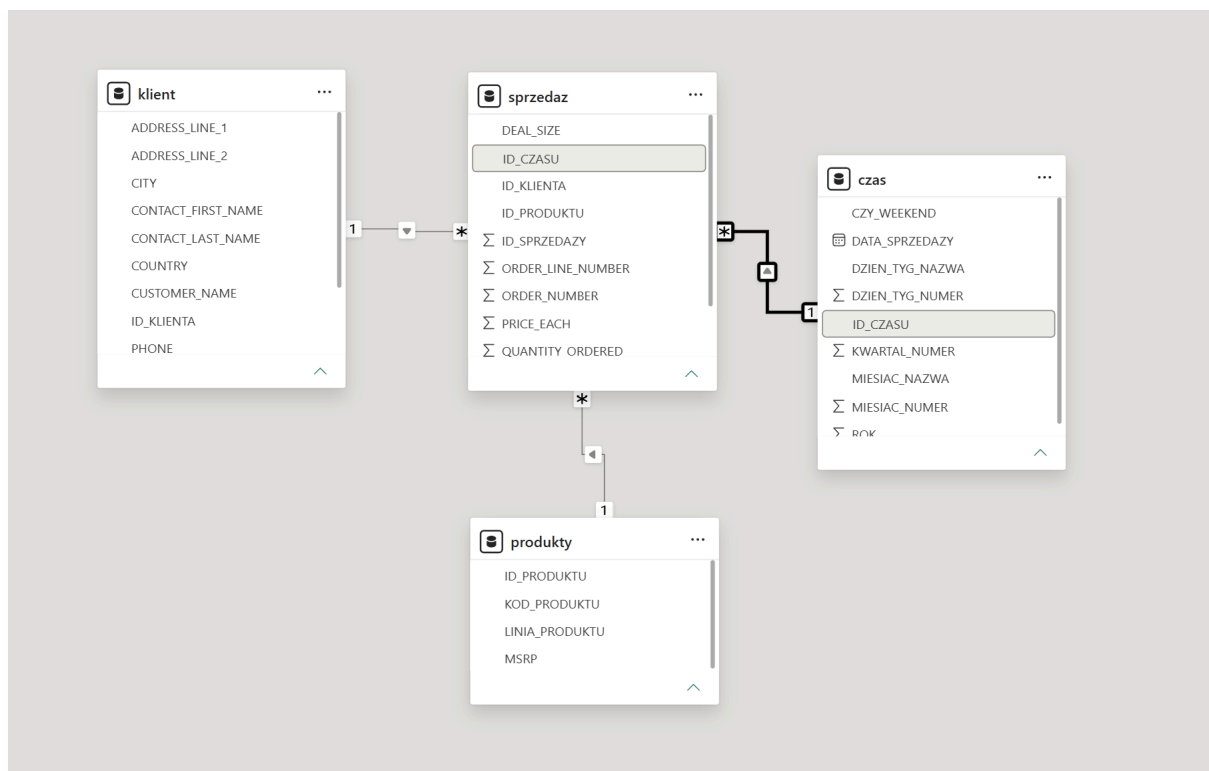
325 SELECT
326     c.rok,
327     p.linia_produkty,
328     SUM(s.sales) AS suma_sprzedaży,
329     RANK() OVER (PARTITION BY c.rok ORDER BY SUM(s.sales) DESC) AS ranking
330 FROM sprzedaz s
331 JOIN czas c ON s.id_czasu = c.id_czasu
332 JOIN produkty p ON s.id_produkty = p.id_produkty
333 GROUP BY c.rok, p.linia_produkty;

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULT SCRIPT OUTPUT SQL HISTO

All rows fetched: 21 in 0.137 seconds

	ROK	LINIA_PRODUKTU	SUMA_SPRZEDAŻY	RANKING
1	2003	Classic Cars	1484785.29	1
2	2003	Vintage Cars	650987.76	2
3	2003	Trucks and Buses	420429.93	3
4	2003	Motorcycles	370895.58	4
5	2003	Planes	272257.6	5
6	2003	Ships	244821.09	6
7	2003	Trains	72802.29	7
8	2004	Classic Cars	1762257.09	1
9	2004	Vintage Cars	911423.77	2
10	2004	Motorcycles	560545.23	3
11	2004	Trucks and Buses	529302.89	4



Schemat relacji modelu danych w Power BI Desktop. Wszystkie relacje zostały zdefiniowane jako jeden do wielu, połączenia wykonane za pomocą kluczy ID_CZASU, ID_KLIENTA oraz ID_PRODUKTU.

```

1 Suma elementów SALES% między miesiącami =
2 IF(
3     ISFILTERED('czas'[DATA_SPRZEDAZY]),
4     ERROR("Szybkie miary analizy czasowej mogą być grupowane i filtrowane tylko w hierarchii dat dostarczonej przez
    usługę Power BI lub podstawowej kolumnie dat."),
5     VAR __PREV_MONTH =
6         CALCULATE(
7             SUM('sprzedaz'[SALES]),
8             DATEADD('czas'[DATA_SPRZEDAZY].[Date], -1, MONTH)
9         )
10    RETURN
11    DIVIDE(SUM('sprzedaz'[SALES]) - __PREV_MONTH, __PREV_MONTH)
12 )

```

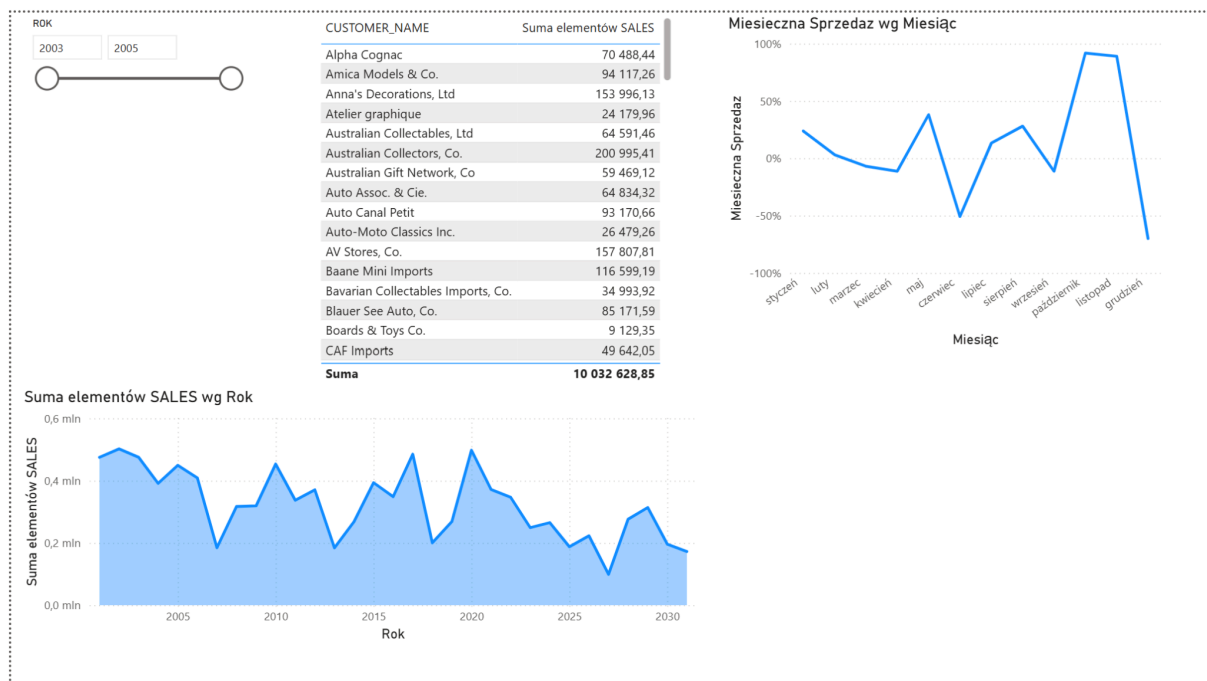
Miara automatycznie oblicza procentową różnicę w sumie wartości sprzedaży sales pomiędzy bieżącym miesiącem a miesiącem bezpośrednio go poprzedzającym.

```

1 Średnia krocząca: Suma elementów SALES =
2 IF(
3     ISFILTERED('czas'[DATA_SPRZEDAZY]),
4     ERROR("Szybkie miary analizy czasowej mogą być grupowane i filtrowane tylko w hierarchii dat dostarczonej przez
    usługę Power BI lub podstawowej kolumnie dat."),
5     VAR __LAST_DATE = ENDOFMONTH('czas'[DATA_SPRZEDAZY].[Date])
6     VAR __DATE_PERIOD =
7         DATESBETWEEN(
8             'czas'[DATA_SPRZEDAZY].[Date],
9             STARTOFMONTH(DATEADD(__LAST_DATE, -3, MONTH)),
10            __LAST_DATE
11        )
12    RETURN
13    AVERAGEX(
14        CALCULATETABLE(
15            SUMMARIZE(
16                VALUES('czas'),
17                'czas'[DATA_SPRZEDAZY].[Rok],
18                'czas'[DATA_SPRZEDAZY].[Numer kwartału],
19                'czas'[DATA_SPRZEDAZY].[Kwartał],
20                'czas'[DATA_SPRZEDAZY].[Numer miesiąca],
21                'czas'[DATA_SPRZEDAZY].[Miesiąc]
22            ),
23            __DATE_PERIOD
24        ),
25        CALCULATE(SUM('sprzedaz'[SALES]), ALL('czas'[DATA_SPRZEDAZY].[Dzień]))
26    )
27 )

```

Miara oblicza średnią wartość sprzedaży sales z ruchomego okna czasowego, które w tym przypadku obejmuje 3 poprzednie miesiące oraz miesiąc bieżący.



Ogólne podsumowanie wyników handlowych przedsiębiorstwa za pomocą czterech elementów:

- Slicer: Oparty na kolumnie rok, filtruje zawartość strony względem roku.
- Table: Prezentuje zestawienie danych nazw z sumą sprzedaży
- Line Chart: Oś X reprezentuje upływ czasu, a oś Y procentowe skoki i spadki obrotów.
- Area Chart: Ukazuje całkowitą wartość sprzedaży względem czasu

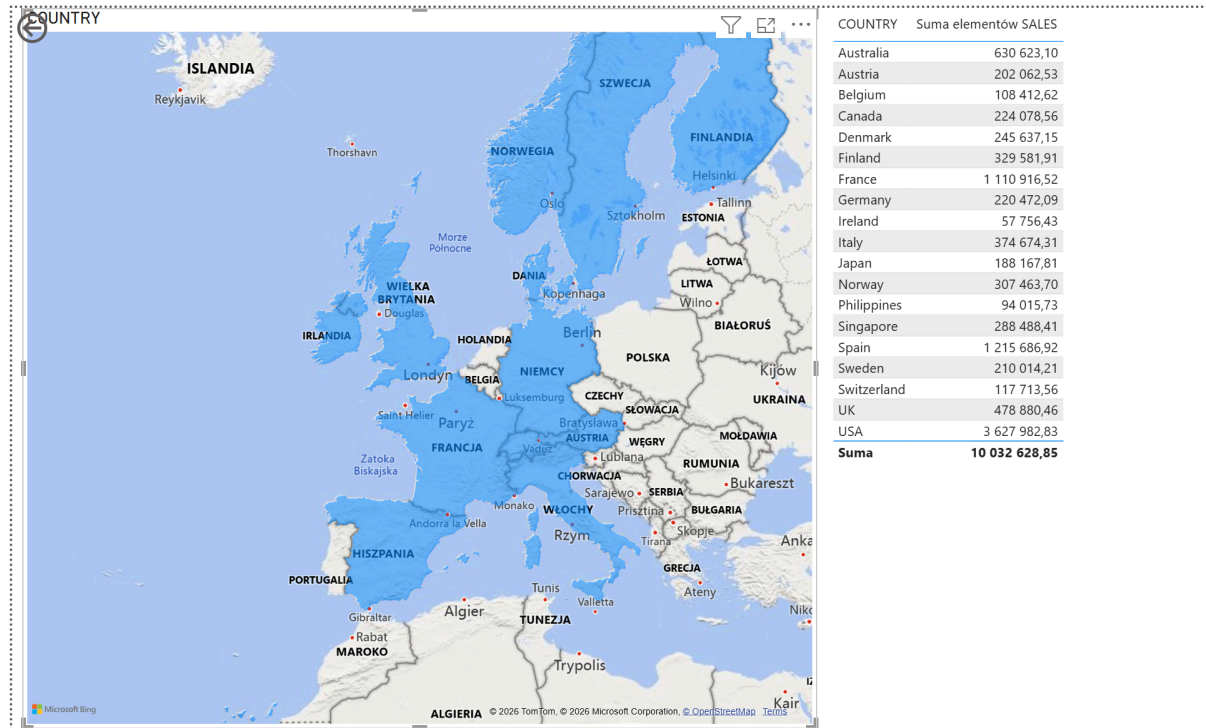


Część skupiona na Produktach posiadająca:

- Bar chart: Wykres słupkowy, który zestawia kategorie produktów z ich całkowitym udziałem w przychodach.

-Scatter Chart: Wykres prezentujący zależności między zachowaniami zakupowymi. Ukazuje relację pomiędzy price_each na osi Y a quantity_ordered na osi X ukazując kody produktów

-Matrix: Ukazuje wartość sprzedaży zależną od roku i lini produktu



Część poświęcona mapie.

-Kartogram: Na podstawie country mapa wyświetla kraje w której notowana była sprzedaż. Daje możliwość filtrowania po kraju aby w tabeli po prawej sprawdzić sprzedaż w danym kraju.

Interpretacja Danych:

Wykres warstwowy oraz liniowy pokazują skok obrotów pod koniec każdego roku, z rekordem w listopadzie, natomiast w pierwszym kwartale roku przykładowo w styczniu możemy zaobserwować spadek sprzedaży.

Wykres słupkowy i Macierz jednoznacznie wskazują, że samochody klasyczne to najlepiej sprzedająca się linia produktów.

Wykres punktowy pokazuje, że większość produktów od początku sprzedawało się w okolicach tysiąca sztuk. Świadczy to o podobnym popycie każdego z produktów z wyjątkiem jednego z nich.

Dzięki geograficznemu ukazaniu danych możemy zobaczyć z jakich krajów zamawiano produkty oraz w którym z nich popyt jest największy. W tym przypadku najwięcej sprzedaży mamy w USA.